- Behavioral tests, sometimes known as *acceptance tests*, specify a feature within your program and are often a user-facing process.
- UI tests focus on the user interface. These usually require specialty tools that simulate human interaction with your software.
- Testing your software from the top down is expensive: every UI interaction interacts with the entire stack. This slows your testing down and causes low-level components to be tested redundantly. The automated testing hierarchy is often visualized as a pyramid, with the majority of tests being at the bottom of the hierarchy, at the unit test level.
- Python's standard library includes the `unittest` module that defines the structure of automated testing.
- The `unittest` module searches for `TestCase` classes, running methods that begin with `test_`. The `TestCase` class also has methods for common setup and tear-down activities required for the test.
- A test runner is responsible for finding tests, executing them, and reporting on whether they have passed.
- Django inherits from Python's `TestCase`, providing additional features specific to the web framework.
- Django's test runner looks for `tests.py` files in the app directories and runs all methods that begin with `test_` inside a class that inherits from `TestCase`.
- You invoke the Django test runner using the `test` management command.
- The `TestCase` class provides a number of "assert" methods that raise an exception if their test condition isn't met. The most commonly used one is `.assertEqual()`.
- The Django `TestCase` class creates a test copy of a database, setting up everything the ORM needs to work with your apps. The database begins empty, so you need to populate Model objects for your tests.
- Each test method gets wrapped in a database transaction, so the actions your test performs on the database get undone after the method completes. This ensures tests start in a known environment and can be run in any order.
- The `.setUp()` and `.setUpClass()` methods get used to create the environment needed for your tests, including populating any ORM Models.
- Django's `TestCase` class has a `.client` object that allows you to authenticate to the system and invoke both GET and POST methods through the URL router.
- The `.client.get()` and `.client.post()` methods return a response object that contains the context dictionary sent to the template engine, the resulting rendered content, and the HTTP status code for the call.
- Your tests should cover as many of the scenarios in your view as possible, including negative cases where errors should be displayed. For example, you can verify a `404` page was displayed by checking the `status_code` field of the response object.